# UNIVERSITÉ D'OTTAWA

uOttawa

Faculté de génie
Faculty of Engineering

Hiver 2026

## École de science informatique et de génie électrique

## **CSI 2532:** Bases de données I

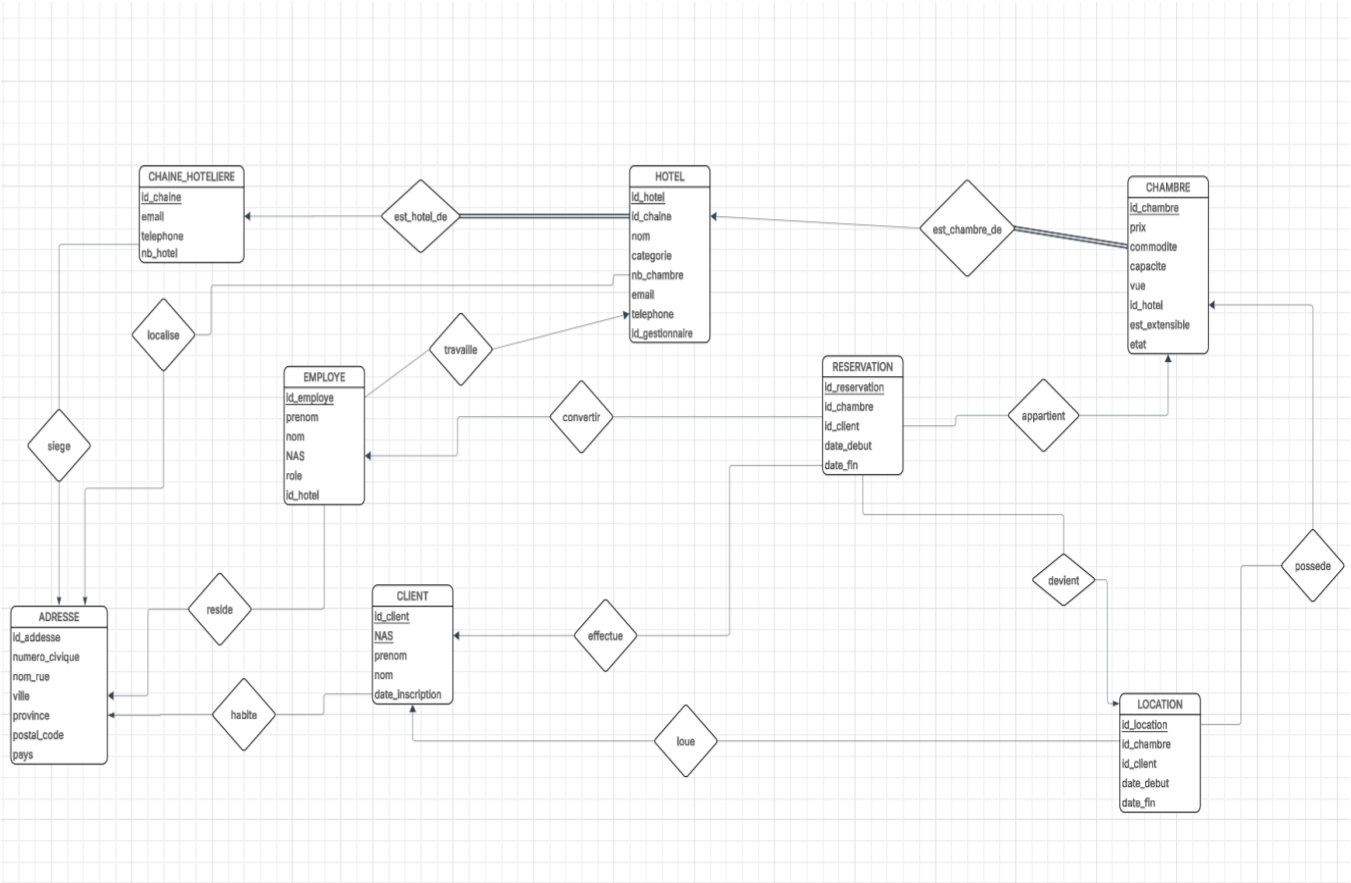
**Projet :** Systemes e-Hotels

**Livrable 1 & Devoir 4:** Modélisation et conception de la base de données

**Date de remise : 3 avril 2026 à 20h00**


**Autheurs:**
**Gilly Dyvan Toukam Mengaptche: 300268205**
**Naomi Kesley Tekeu : 300261358**
**Frank Martial Penka Nghoko**
**Antoine Missoup Konga**

# 1. Diagramme ER — avec brève justification




## Entités retenues

| Entité | Attributs |
|---|---|
| **CHAINE_HOTELIERE** | id_chaine, nom, email, telephone, nb_hotel |

| | |
|---|---|
| **HOTEL** | id_hotel, id_chaine, id_gestionnaire, nom, email, categorie, nb_chambre, telephone, id_adresse |
| **CHAMBRE** | id_chambre, id_hotel, prix, commodite, capacite, vue, est_extensible, etat |
| **CLIENT** | id_client, NAS, prenom, nom, date_inscription, id_adresse |
| **EMPLOYE** | id_employe, id_hotel, prenom, nom, NAS, role, id_adresse |
| **RESERVATION** | id_reservation, id_chambre, id_client, date_debut, date_fin |
| **LOCATION** | id_location, id_client, id_employe, id_reservation, id_chambre, date_debut, date_fin |
| **ADRESSE** | id_adresse, numero_civique, nom_rue, ville, province, postal_code, pays |

## Relations principales

- Une **chaîne hôtelière** possède plusieurs **hôtels**
- Un **hôtel** contient plusieurs **chambres**
- Un **hôtel** emploie plusieurs **employés**
- Un **client** effectue plusieurs **réservations**
- Un **client** peut avoir plusieurs **locations**
- Une **réservation** concerne une **chambre**
- Une **location** concerne une **chambre**
- Une **réservation** peut devenir une **location**
- Un **employé** est responsable de la conversion ou de la location

- Une **adresse** peut être associée à une chaîne, un hôtel, un client ou un employé

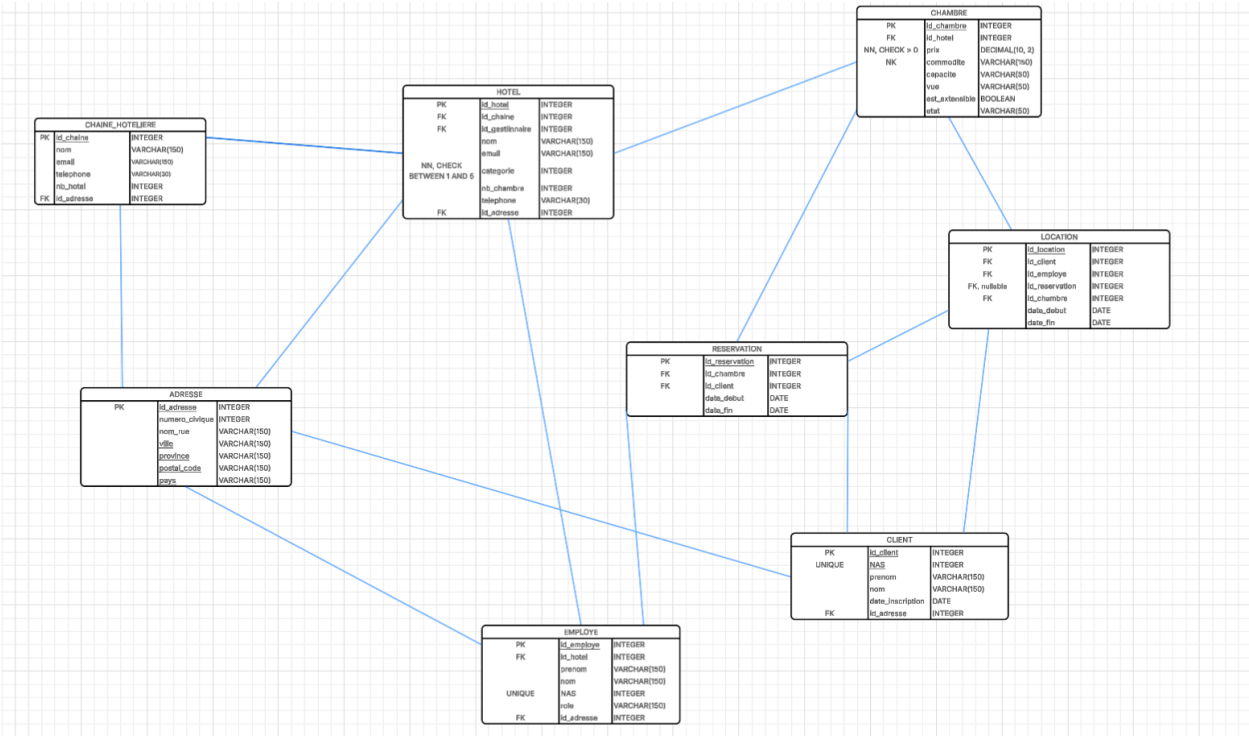
## Brève justification du diagramme ER

Le diagramme ER a été construit à partir des exigences du système e-**Hôtels**. Le sujet impose la gestion des **chaînes hôtelières, hôtels, chambres, clients, employés, réservations et locations**, ainsi que les liens hiérarchiques **chaîne → hôtel → chambre**. Il impose aussi que **chaque hôtel appartienne à une chaîne**, qu'**une chambre ne puisse exister sans hôtel**, qu'**un client puisse réserver ou louer directement**, et que **chaque hôtel ait un gestionnaire**.

L'entité **ADRESSE** a été séparée pour éviter la duplication d'attributs d'adresse dans plusieurs entités. Le choix d'inclure **id_gestionnaire** dans HOTEL permet de représenter explicitement la contrainte selon laquelle chaque hôtel doit avoir un gestionnaire. La distinction entre **RESERVATION** et **LOCATION** est nécessaire, car l'énoncé précise qu'une réservation peut être convertie en location lors du check-in, mais qu'une location peut aussi être faite directement sans réservation.

Dans cette version du modèle, certains attributs comme **`email`**, **`telephone`** et **`commodité`** ont été représentés sous forme simple afin de garder un schéma plus lisible pour le livrable 1. Une version plus normalisée pourrait séparer ces éléments dans des tables additionnelles, notamment pour gérer plusieurs emails, plusieurs téléphones ou plusieurs commodités par chambre.

# 2. Schéma de base de données relationnelle




## Brève justification du schéma relationnel

Le schéma relationnel a été obtenu par transformation du diagramme ER. Chaque entité forte du modèle conceptuel devient une table relationnelle, avec une **clé primaire** pour identifier chaque tuple. Les relations **1:N** sont traduites par des **clés étrangères** placées du côté N. Ainsi, HOTEL contient id_chaine, CHAMBRE contient id_hotel, EMPLOYE contient id_hotel, et RESERVATION contient id_client et id_chambre.

La table LOCATION contient une clé étrangère nullable id_reservation afin de refléter le fait qu'une location peut soit provenir d'une réservation convertie, soit être créée directement sans réservation préalable. Ce choix respecte le cahier des charges. La table ADRESSE a été conservée séparément pour factoriser les informations d'adresse utilisées par plusieurs entités.

---

# 3. Contraintes définies — avec justification pour chacune

L’énoncé demande explicitement d’inclure des **clés primaires, clés étrangères, contraintes de domaine et contraintes personnalisées**.

## 3.1 Clés primaires

### ADRESSE.id_adresse

Justification : identifie de manière unique chaque adresse.

### CHAINE_HOTELIERE.id_chaine

Justification : identifie de manière unique chaque chaîne hôtelière.

### HOTEL.id_hotel

Justification : identifie de manière unique chaque hôtel.

### CHAMBRE.id_chambre

Justification : identifie de manière unique chaque chambre.

### CLIENT.id_client

Justification : identifie de manière unique chaque client.

### EMPLOYE.id_employe

Justification : identifie de manière unique chaque employé.

### RESERVATION.id_reservation

Justification : identifie de manière unique chaque réservation.

### LOCATION.id_location

Justification : identifie de manière unique chaque location.

---

## 3.2 Clés étrangères

### CHAINE_HOTELIERE.id_adresse -> ADRESSE(id_adresse)

Justification : une chaîne possède une adresse de siège social, comme demandé dans le sujet.

### HOTEL.id_chaine -> CHAINE_HOTELIERE(id_chaine)

Justification : un hôtel appartient obligatoirement à une chaîne. Cela traduit la règle métier « un hôtel ne peut exister sans chaîne ».

### HOTEL.id_gestionnaire -> EMPLOYE(id_employe)

Justification : chaque hôtel doit avoir un gestionnaire. Cette clé étrangère permet de modéliser cette exigence explicitement.

### HOTEL.id_adresse -> ADRESSE(id_adresse)

Justification : chaque hôtel possède une adresse.

### CHAMBRE.id_hotel -> HOTEL(id_hotel)

Justification : une chambre appartient obligatoirement à un hôtel. Cela traduit la règle métier « une chambre ne peut exister sans hôtel ».

### CLIENT.id_adresse -> ADRESSE(id_adresse)

Justification : chaque client possède une adresse.

### EMPLOYE.id_hotel -> HOTEL(id_hotel)

Justification : chaque employé travaille dans un hôtel donné.

### EMPLOYE.id_adresse -> ADRESSE(id_adresse)

Justification : chaque employé possède une adresse.

### RESERVATION.id_client -> CLIENT(id_client)

Justification : une réservation est effectuée par un client.

### RESERVATION.id_chambre -> CHAMBRE(id_chambre)

Justification : une réservation concerne une chambre précise.

### LOCATION.id_client -> CLIENT(id_client)

Justification : une location est associée à un client.

### LOCATION.id_employe -> EMPLOYE(id_employe)

Justification : le sujet précise qu'un employé est responsable de la conversion ou de la location.

### LOCATION.id_reservation -> RESERVATION(id_reservation) nullable

Justification : une location peut provenir d'une réservation, mais peut aussi être créée directement sans réservation préalable.

### LOCATION.id_chambre -> CHAMBRE(id_chambre)

Justification : une location porte sur une chambre donnée.

---

## 3.3 Contraintes de domaine

### HOTEL.categorie CHECK (categorie BETWEEN 1 AND 5)

Justification : l'énoncé indique une catégorie d'hôtel de type 1 à 5 étoiles. Cette contrainte limite les valeurs possibles à cet intervalle.

### CHAMBRE.prix CHECK (prix > 0)

Justification : le prix d'une chambre doit être strictement positif.

### CLIENT.NAS UNIQUE

Justification : le NAS doit être unique pour identifier correctement chaque client et éviter les doublons.

### EMPLOYE.NAS UNIQUE

Justification : le NAS doit aussi être unique pour chaque employé.

### LOCATION.id_reservation nullable

Justification : permet la location directe sans réservation, exigée par le sujet.

---

## 3.4 Contraintes personnalisées

### Contrainte 1 — Chaque hôtel doit avoir un gestionnaire

Justification : c'est une exigence explicite du sujet. Elle est modélisée via HOTEL.id_gestionnaire.

### Contrainte 2 — Le gestionnaire d'un hôtel doit être un employé de ce même hôtel

Justification : cette règle évite qu'un hôtel référence comme gestionnaire un employé d'un autre hôtel. C'est une contrainte métier complémentaire.

### Contrainte 3 — date_debut < date_fin dans RESERVATION

Justification : garantit la cohérence temporelle d'une réservation.

### Contrainte 4 — date_debut < date_fin dans LOCATION

Justification : garantit la cohérence temporelle d'une location.

### Contrainte 5 — Conservation de l'historique

**Justification** : l'énoncé demande de stocker l'historique des réservations et des locations, et précise que les données archivées doivent rester même si certaines entités sont supprimées. Cette contrainte sera surtout gérée au niveau logique ou via une stratégie d'archivage lors de l'implémentation.